

mugshot_removal — Combined Source

FILE: README.md

Mugshot Removal Tool (multi-state)

Search, draft, send, and track booking-photo removal requests across states with a fee-prohibition/removal statute. Each request is generated and sent as the client's own first-person statutory request (the client is the "From" address and the signer) – not as a third party representing them.

Supported states

See `statutes.py`. Each entry is either `"verified"` (citation, deadline, and penalty all confirmed) or `"partial"` (citation and fee-prohibition confirmed; deadline/penalty not confirmed and left blank rather than guessed – the generated letter says so explicitly and asks the operator to verify current statutory text before sending). Currently: FL (verified), CA, TX, GA, UT, OR, CO, IL, WY (partial). Run `python main.py list-states` for the full data. Adding a new state means adding a real, sourced entry to `statutes.py` – not extrapolating from another state's numbers.

Setup

...

pip install -r requirements.txt

...

****Search**** (Google Custom Search API):

1. <https://programmablesearchengine.google.com/> → create a search engine scoped to the entire web → copy its Search Engine ID (`cx`).
2. <https://console.cloud.google.com> → enable "Custom Search API" → create an API key.

****Send**** (SMTP): use the client's own email account (or an account they've authorized you to send from as them), with an app password if the provider requires one (e.g. Gmail).

****Track**** (optional, Google Sheets): same service-account setup as `lead\_extraction/` – see that folder's README for the steps. Share the tracking sheet with the service account's `client\_email`.

Usage

Find candidate pages:

...

python main.py search --client-name "Jane Public" --api-key KEY --cx CX_ID

...

Preview a letter without sending (defaults to `--state FL`):

...

python main.py draft --client-name "Jane Public" --client-contact "jane@example.com" \
--target-url "https://example-mugshot-site.com/jane-public" \
--state FL \
--booking-date "2026-08-19" --arresting-agency "Palm Beach County Sheriff's Office"

...

****Before sending anything real****, get the client's written authorization to send from their name/email. Generate the form:

...

python main.py consent-form --client-name "Jane Public" \
--client-email "jane@example.com" \
--target-url "https://example-mugshot-site.com/jane-public" \
--state FL

...

Have the client sign and return it, and keep it on file. `send` will not run without `--consent-date` set to the date they actually signed it – a

future or garbage date is rejected outright, not just logged.

Send it and log to a tracker sheet:

```
python main.py send --client-name "Jane Public" --client-email "jane@example.com" \
--target-url "https://example-mugshot-site.com/jane-public" \
--to-email "abuse@example-mugshot-site.com" \
--state FL \
--consent-date "2026-08-25" \
--booking-date "2026-08-19" --arresting-agency "Palm Beach County Sheriff's Office" \
--smtp-host smtp.gmail.com --smtp-port 465 \
--smtp-username jane@example.com --smtp-password APP_PASSWORD \
--sheet-id YOUR_SHEET_ID
```

Check which requests are past their 10-day deadline with no removal – this also looks up each overdue site's domain registrant and hosting provider (WHOIS + IP RDAP) and writes that into the tracker sheet:

```
python main.py check-overdue --sheet-id YOUR_SHEET_ID
```

That's the point at which Fla. Stat. § 901.43 lets the client seek an injunction, a \$1,000/day penalty, and attorney fees – that step itself still needs an actual attorney to file. This tool's job stops at handing that attorney the request's overdue status plus who to name/contact (registrant and hosting org, where WHOIS/RDAP data isn't privacy-shielded).

Look up registrant/hosting info for a single URL directly:

```
python main.py lookup-owner --target-url "https://example-mugshot-site.com/jane-public"
```

Scope

- Each letter is the client's own request, in their own name – this tool doesn't send correspondence asserting representation of the client.
- `send` is gated on a recorded, past-dated consent date; it will not send without one.
- No claim of copyright ownership over the photograph is made anywhere in the letter; the basis is the client's own right under the applicable statute.
- Finding pages uses Google's Custom Search API, not scraped search results.

FILE: requirements.txt

```
google-api-python-client>=2.100.0  
gsread>=6.0.0  
google-auth>=2.28.0  
python-whois>=0.9.4  
ipwhois>=1.2.0
```

FILE: statutes.py

```
"""State-by-state booking-photograph removal statutes.
```

```
Only includes fields actually confirmed via research for this build. Where a
removal deadline or penalty wasn't confirmed, it's left as None rather than
guessed -- a demand letter citing a wrong number is worse than one that
cites the statute correctly and states the request without a fabricated
figure. VERIFY current statutory text before relying on any entry here for
real correspondence; state legislatures amend and renumber these regularly.
```

```
confidence:
```

```
    "verified"    -> citation, deadline, and penalty confirmed from research
    "partial"     -> citation and fee-prohibition confirmed; deadline/penalty
                    not confirmed, left as None
```

```
"""
```

```
STATE_STATUTES = {
    "FL": {
        "name": "Florida",
        "citation": "Fla. Stat. § 901.43",
        "fee_prohibited": True,
        "removal_deadline_days": 10,
        "penalty_description": "civil penalty up to $1,000/day of noncompliance with an injunction, plus attorney fees",
        "confidence": "verified",
    },
    "CA": {
        "name": "California",
        "citation": "Cal. Civ. Code § 1798.91.1",
        "fee_prohibited": True,
        "removal_deadline_days": None,
        "penalty_description": None,
        "confidence": "partial",
    },
    "TX": {
        "name": "Texas",
        "citation": "Tex. Bus. & Com. Code ch. 109",
        "fee_prohibited": True,
        "removal_deadline_days": 45,
        "penalty_description": "civil action for noncompliance; damages and other legal remedies per statute",
        "confidence": "partial",
    },
    "GA": {
        "name": "Georgia",
        "citation": "O.C.G.A. § 35-1-19 (HB 845)",
        "fee_prohibited": True,
        "removal_deadline_days": None,
        "penalty_description": None,
        "confidence": "partial",
    },
    "UT": {
        "name": "Utah",
        "citation": "Utah Code § 17-22-30",
        "fee_prohibited": True,
        "removal_deadline_days": None,
        "penalty_description": None,
        "confidence": "partial",
    },
    "OR": {
        "name": "Oregon",
        "citation": "Or. Rev. Stat. § 30.835",
        "fee_prohibited": True,
        "removal_deadline_days": None,
        "penalty_description": None,
        "confidence": "partial",
    },
    "CO": {
        "name": "Colorado",
        "citation": "Colo. Rev. Stat. § 18-12-105.7",
```

```

        "fee_prohibited": True,
        "removal_deadline_days": None,
        "penalty_description": None,
        "confidence": "partial",
    },
    "IL": {
        "name": "Illinois",
        "citation": "730 ILCS 5/5-4-7",
        "fee_prohibited": True,
        "removal_deadline_days": None,
        "penalty_description": None,
        "confidence": "partial",
    },
    "WY": {
        "name": "Wyoming",
        "citation": "Wyoming booking-photograph removal statute (2021 session laws)",
        "fee_prohibited": True,
        "removal_deadline_days": None,
        "penalty_description": None,
        "confidence": "partial",
    },
}

```

```

def get_statute(state_code: str) -> dict:
    state_code = state_code.upper()
    if state_code not in STATE_STATUTES:
        raise KeyError(
            f"No statute entry for '{state_code}'. Only states actually researched for "
            f"this tool are included: {'', ' '.join(STATE_STATUTES)}. Add an entry to "
            f"statutes.py (with real citation/confidence) before using a new state."
        )
    return STATE_STATUTES[state_code]

```

FILE: consent.py

```
"""Client authorization record: written permission for a removal request to
be sent from the client's own name and email address.
```

```
This is what makes a letter genuinely the client's own request rather than
someone else acting for them -- it needs to exist, in writing, before any
real letter goes out. generate_consent_form() gives the client something to
sign/return; the CLI's send command refuses to run without a recorded
consent date (see main.py).
"""
```

```
from datetime import date
```

```
CONSENT_FORM_TEMPLATE = """\
AUTHORIZATION TO SEND BOOKING PHOTOGRAPH REMOVAL REQUEST
```

```
I, {client_name}, authorize the sending of a written request for removal of
my arrest booking photograph, under {citation}, from the following email
address on my behalf and in my name:
```

```
{client_email}
```

```
Target page to be addressed: {target_url}
```

```
I understand this request will be sent as my own first-person statutory
request, signed with my name, and that I am the person entitled to enforce
the rights described in it.
```

```
Signature: _____
```

```
Printed name: {client_name}
```

```
Date: _____
```

```
"""
```

```
def generate_consent_form(client_name: str, client_email: str, target_url: str, citation: str) -> str:
    return CONSENT_FORM_TEMPLATE.format(
        client_name=client_name,
        client_email=client_email,
        target_url=target_url,
        citation=citation,
    )
```

```
def validate_consent_date(consent_date_str: str) -> date:
    """Parse and sanity-check a claimed consent date. Raises ValueError if
    the string isn't a real date or is in the future."""
    parsed = date.fromisoformat(consent_date_str)
    if parsed > date.today():
        raise ValueError(f"Consent date {consent_date_str} is in the future.")
    return parsed
```

FILE: search.py

```
"""Find pages that appear to host a given client's booking photo.
```

Uses Google's official Custom Search JSON API (not scraping google.com search result pages directly, which is against Google's terms). Requires a Custom Search Engine configured to search the whole web, plus an API key.

Setup:

1. <https://programmablesearchengine.google.com/> -> create a search engine, set it to search the entire web, copy its Search Engine ID (cx).
2. <https://console.cloud.google.com> -> enable "Custom Search API", create an API key.

```
"""
```

```
from googleapiclient.discovery import build
```

```
def find_mugshot_pages(client_name: str, api_key: str, cx: str, num_results: int = 20) -> list[dict]:
    """Return a list of {title, link, snippet} results for likely mugshot pages."""
    service = build("customsearch", "v1", developerKey=api_key)
    query = f'"{client_name}" mugshot OR arrest OR booking'

    results = []
    start = 1
    while len(results) < num_results:
        batch_size = min(10, num_results - len(results))
        response = service.cse().list(q=query, cx=cx, start=start, num=batch_size).execute()
        items = response.get("items", [])
        if not items:
            break
        for item in items:
            results.append({
                "title": item.get("title", ""),
                "link": item.get("link", ""),
                "snippet": item.get("snippet", ""),
            })
        start += batch_size

    return results
```

FILE: draft.py

```
"""Generate a booking-photo removal request under a given state's statute.

Written in the client's own first-person voice, asserting the client's own
statutory right -- not as a representative acting on the client's behalf,
and not asserting ownership of the photograph itself.
"""

from datetime import date, timedelta

from statutes import get_statute

TEMPLATE_WITH_DEADLINE = """\
Subject: Request for Removal of Booking Photograph Pursuant to {citation}

To Whom It May Concern:

My name is {client_name}. I am writing to request the removal of my arrest
booking photograph, which currently appears on your website at the following
URL:

{target_url}

Under {citation}, a person or entity that publishes an arrest booking
photograph may not charge a fee to remove it, and must remove it within
{deadline_days} calendar days of receiving a written request from the person
depicted. This letter constitutes that written request.

Booking date: {booking_date}
Arresting agency: {arresting_agency}

Please confirm removal in writing. If the photograph is not removed within
{deadline_days} calendar days of the date of this request, I am entitled
under {citation} to pursue: {penalty_description}.

Sincerely,
{client_name}
{client_contact}

Date of this request: {request_date}
Deadline for removal: {deadline}
"""

TEMPLATE_NO_DEADLINE = """\
Subject: Request for Removal of Booking Photograph Pursuant to {citation}

To Whom It May Concern:

My name is {client_name}. I am writing to request the removal of my arrest
booking photograph, which currently appears on your website at the following
URL:

{target_url}

Under {citation}, a person or entity that publishes an arrest booking
photograph may not charge a fee to remove, correct, or refrain from
publishing it. This letter constitutes my written request for removal.

Booking date: {booking_date}
Arresting agency: {arresting_agency}

Please confirm removal in writing. [NOTE TO OPERATOR: this state's specific
removal deadline and noncompliance penalty were not confirmed for this tool
-- verify current statutory text for {state_name} before sending, and add
the specific deadline/penalty language before relying on this letter.]

Sincerely,
{client_name}
{client_contact}
```



```
Date of this request: {request_date}
"""
```

```
def generate_letter(
    client_name: str,
    client_contact: str,
    target_url: str,
    state: str = "FL",
    booking_date: str = "",
    arresting_agency: str = "",
    request_date: date | None = None,
) -> str:
    statute = get_statute(state)
    request_date = request_date or date.today()

    common = dict(
        citation=statute["citation"],
        client_name=client_name,
        client_contact=client_contact,
        target_url=target_url,
        booking_date=booking_date or "N/A",
        arresting_agency=arresting_agency or "N/A",
        request_date=request_date.isoformat(),
        state_name=statute["name"],
    )

    if statute["removal_deadline_days"] is not None:
        deadline = request_date + timedelta(days=statute["removal_deadline_days"])
        return TEMPLATE_WITH_DEADLINE.format(
            **common,
            deadline_days=statute["removal_deadline_days"],
            penalty_description=statute["penalty_description"] or "the remedies available under this statute",
            deadline=deadline.isoformat(),
        )

    return TEMPLATE_NO_DEADLINE.format(**common)
```

FILE: send.py

"""Send a removal request by email, from the client's own email account.

The message is sent as the client (From: the client's address), not as a third party representing them -- consistent with the letter being the client's own first-person statutory request.

"""

```
import smtplib
```

```
from email.mime.text import MIMEText
```

```
def send_removal_request(
```

```
    smtp_host: str,
```

```
    smtp_port: int,
```

```
    smtp_username: str,
```

```
    smtp_password: str,
```

```
    client_email: str,
```

```
    to_email: str,
```

```
    letter_body: str,
```

```
) -> None:
```

```
    subject, _, body = letter_body.partition("\n\n")
```

```
    subject = subject.replace("Subject: ", "", 1)
```

```
    msg = MIMEText(body)
```

```
    msg["Subject"] = subject
```

```
    msg["From"] = client_email
```

```
    msg["To"] = to_email
```

```
    with smtplib.SMTP_SSL(smtp_host, smtp_port) as server:
```

```
        server.login(smtp_username, smtp_password)
```

```
        server.sendmail(client_email, [to_email], msg.as_string())
```

FILE: tracker.py

```
"""Track sent removal requests and their compliance deadline in a Google
Sheet, across whichever state's statute applies to each request."""
from datetime import date, timedelta
```

```
import gspread
from google.oauth2.service_account import Credentials
```

```
SCOPES = ["https://www.googleapis.com/auth/spreadsheets"]
```

```
HEADER = [
    "Client Name",
    "State",
    "Target URL",
    "Contact Email",
    "Consent Date",
    "Request Sent Date",
    "Deadline",
    "Status",
    "Notes",
    "Registrant Org",
    "Registrant Name",
    "Registrant Email",
    "Registrar",
    "Hosting Org",
    "Hosting Abuse Email",
]
```

```
def open_tracker(sheet_id: str, creds_path: str, worksheet_name: str = "Removal Requests"):
    creds = Credentials.from_service_account_file(creds_path, scopes=SCOPES)
    client = gspread.authorize(creds)
    spreadsheet = client.open_by_key(sheet_id)
    try:
        ws = spreadsheet.worksheet(worksheet_name)
    except gspread.WorksheetNotFound:
        ws = spreadsheet.add_worksheet(title=worksheet_name, rows=1000, cols=len(HEADER))
    if ws.row_values(1) != HEADER:
        ws.update(range_name="A1", values=[HEADER])
    return ws
```

```
def log_request(ws, client_name: str, state: str, target_url: str, contact_email: str,
                consent_date: date, deadline_days: int | None, sent_date: date | None = None,
                status: str = "Pending", notes: str = ""):
    """consent_date is required: a request must not be logged (or sent --
    see main.py) without a recorded date the client authorized it."""
    sent_date = sent_date or date.today()
    deadline = (sent_date + timedelta(days=deadline_days)).isoformat() if deadline_days else "N/A (verify statute)"
    ws.append_row([
        client_name,
        state,
        target_url,
        contact_email,
        consent_date.isoformat(),
        sent_date.isoformat(),
        deadline,
        status,
        notes,
        "", "", "", "", "", "",
    ], value_input_option="USER_ENTERED")
```

```
def list_overdue(ws) -> list[dict]:
    """Return rows whose deadline has passed and status is still Pending.
    Rows with a non-date deadline (unconfirmed-statute states) are skipped
    rather than guessed at."""
    today = date.today()
```

```

rows = ws.get_all_records()
overdue = []
for row in rows:
    if row.get("Status") != "Pending":
        continue
    try:
        deadline = date.fromisoformat(row["Deadline"])
    except (ValueError, KeyError):
        continue
    if deadline < today:
        overdue.append(row)
return overdue

def record_owner_info(ws, target_url: str, lookup_result: dict):
    """Write registrant/hosting lookup results into the row matching
    target_url (the most recent matching row if there are several)."""
    domain_info = lookup_result.get("domain_info", {})
    hosting_info = lookup_result.get("hosting_info", {})

    cell = None
    matches = ws.findall(target_url)
    if matches:
        cell = matches[-1]
    if cell is None:
        return False

    row = cell.row
    ws.update(range_name=f"J{row}:O{row}", values=[
        domain_info.get("registrant_org") or "",
        domain_info.get("registrant_name") or "",
        domain_info.get("registrant_email") or "",
        domain_info.get("registrar") or "",
        hosting_info.get("hosting_org") or "",
        hosting_info.get("abuse_email") or "",
    ])
    return True

```

FILE: owner_lookup.py

"""Look up domain registrant and hosting provider info for a target URL.

This is pre-litigation research (who to name/serve, who to contact) for handing off to an attorney once a request has gone past its 10-day deadline under Fla. Stat. § 901.43 -- it does not send anything or make any legal claim itself.

"""

```
import socket
from urllib.parse import urlparse
```

```
import whois
from ipwhois import IPWhois
```

```
def _domain_from_url(url: str) -> str:
    parsed = urlparse(url if "://" in url else f"http://{url}")
    return parsed.netloc or parsed.path
```

```
def get_domain_info(url: str) -> dict:
    """Registrant/registrar info from WHOIS. Many domains use privacy
    proxies, in which case registrant fields will show the proxy service
    rather than the underlying owner -- that's the WHOIS record's own
    limitation, not something this function can see past."""
    domain = _domain_from_url(url)
    try:
        w = whois.whois(domain)
    except Exception as e:
        return {"domain": domain, "error": str(e)}

    return {
        "domain": domain,
        "registrant_org": _first(w.get("org")),
        "registrant_name": _first(w.get("name")),
        "registrant_email": _first(w.get("emails")),
        "registrar": _first(w.get("registrar")),
        "creation_date": _stringify(_first(w.get("creation_date"))),
        "country": _first(w.get("country")),
    }
```

```
def get_hosting_info(url: str) -> dict:
    """Resolve the domain to an IP and look up the hosting network/ASN via
    IP WHOIS -- this identifies the hosting provider, which matters
    separately from the domain registrant for notice/safe-harbor purposes
    (the host, not just the site owner, can be put on notice)."""
    domain = _domain_from_url(url)
    try:
        ip = socket.gethostbyname(domain)
    except Exception as e:
        return {"domain": domain, "error": f"DNS resolution failed: {e}"}

    try:
        ipw = IPWhois(ip)
        result = ipw.lookup_rdap()
    except Exception as e:
        return {"domain": domain, "ip": ip, "error": str(e)}

    network = result.get("network", {})
    return {
        "domain": domain,
        "ip": ip,
        "hosting_org": network.get("name") or result.get("asn_description"),
        "asn": result.get("asn"),
        "asn_description": result.get("asn_description"),
        "abuse_email": _extract_abuse_email(result),
    }
```

```
def _extract_abuse_email(rdap_result: dict) -> str:
    for entity in rdap_result.get("objects", {}).values():
        roles = entity.get("roles") or []
        if "abuse" in roles:
            contact = entity.get("contact") or {}
            emails = contact.get("email") or []
            if emails:
                return emails[0].get("value", "")
    return ""
```

```
def _first(value):
    if isinstance(value, list):
        return value[0] if value else None
    return value
```

```
def _stringify(value):
    return str(value) if value is not None else None
```

```
def full_lookup(url: str) -> dict:
    return {
        "domain_info": get_domain_info(url),
        "hosting_info": get_hosting_info(url),
    }
```

FILE: main.py

```
#!/usr/bin/env python3
"""CLI for the multi-state mugshot removal workflow: search, draft, send, track."""
import argparse
import json

from datetime import date

from consent import generate_consent_form, validate_consent_date
from draft import generate_letter
from owner_lookup import full_lookup
from search import find_mugshot_pages
from send import send_removal_request
from statutes import STATE_STATUTES, get_statute
from tracker import list_overdue, log_request, open_tracker, record_owner_info

def cmd_search(args):
    results = find_mugshot_pages(args.client_name, args.api_key, args.cx, args.num_results)
    print(json.dumps(results, indent=2))

def cmd_draft(args):
    letter = generate_letter(
        client_name=args.client_name,
        client_contact=args.client_contact,
        target_url=args.target_url,
        state=args.state,
        booking_date=args.booking_date,
        arresting_agency=args.arresting_agency,
    )
    print(letter)

def cmd_send(args):
    # Refuse to send without a recorded consent date -- this is the
    # enforcement point for "get written permission and keep it on file"
    # rather than something left to operator memory.
    try:
        consent_date = validate_consent_date(args.consent_date)
    except ValueError as e:
        raise SystemExit(
            f"Refusing to send: {e}\n"
            f"--consent-date must be the date the client actually signed/returned "
            f"their authorization (see `consent-form` command). Not today's date "
            f"unless that's genuinely when they signed it."
        )

    letter = generate_letter(
        client_name=args.client_name,
        client_contact=args.client_email,
        target_url=args.target_url,
        state=args.state,
        booking_date=args.booking_date,
        arresting_agency=args.arresting_agency,
    )
    send_removal_request(
        smtp_host=args.smtp_host,
        smtp_port=args.smtp_port,
        smtp_username=args.smtp_username,
        smtp_password=args.smtp_password,
        client_email=args.client_email,
        to_email=args.to_email,
        letter_body=letter,
    )
    print(f"Sent to {args.to_email}")

    if args.sheet_id:
```

```

statute = get_statute(args.state)
ws = open_tracker(args.sheet_id, args.creds)
log_request(ws, args.client_name, args.state, args.target_url, args.to_email,
            consent_date=consent_date, deadline_days=statute["removal_deadline_days"])
print("Logged to tracker sheet.")

def cmd_consent_form(args):
    statute = get_statute(args.state)
    print(generate_consent_form(args.client_name, args.client_email, args.target_url, statute["citation"]))

def cmd_check_overdue(args):
    ws = open_tracker(args.sheet_id, args.creds)
    overdue = list_overdue(ws)
    for row in overdue:
        result = full_lookup(row["Target URL"])
        record_owner_info(ws, row["Target URL"], result)
        row["owner_lookup"] = result
    print(json.dumps(overdue, indent=2))

def cmd_lookup_owner(args):
    print(json.dumps(full_lookup(args.target_url), indent=2))

def cmd_list_states(args):
    print(json.dumps(STATE_STATUTES, indent=2))

def main():
    parser = argparse.ArgumentParser(description=__doc__)
    sub = parser.add_subparsers(dest="command", required=True)

    p_search = sub.add_parser("search", help="Find pages likely hosting a client's mugshot")
    p_search.add_argument("--client-name", required=True)
    p_search.add_argument("--api-key", required=True, help="Google Custom Search API key")
    p_search.add_argument("--cx", required=True, help="Custom Search Engine ID")
    p_search.add_argument("--num-results", type=int, default=20)
    p_search.set_defaults(func=cmd_search)

    p_draft = sub.add_parser("draft", help="Generate a removal request letter")
    p_draft.add_argument("--client-name", required=True)
    p_draft.add_argument("--client-contact", required=True)
    p_draft.add_argument("--target-url", required=True)
    p_draft.add_argument("--state", default="FL", help="Two-letter state code (see list-states)")
    p_draft.add_argument("--booking-date", default="")
    p_draft.add_argument("--arresting-agency", default="")
    p_draft.set_defaults(func=cmd_draft)

    p_send = sub.add_parser("send", help="Draft, send, and log a removal request")
    p_send.add_argument("--client-name", required=True)
    p_send.add_argument("--client-email", required=True)
    p_send.add_argument("--target-url", required=True)
    p_send.add_argument("--to-email", required=True, help="Site's contact/abuse email")
    p_send.add_argument("--state", default="FL", help="Two-letter state code (see list-states)")
    p_send.add_argument("--booking-date", default="")
    p_send.add_argument("--arresting-agency", default="")
    p_send.add_argument("--smtp-host", required=True)
    p_send.add_argument("--smtp-port", type=int, default=465)
    p_send.add_argument("--smtp-username", required=True)
    p_send.add_argument("--smtp-password", required=True)
    p_send.add_argument("--sheet-id", default=None, help="Optional: log to tracker sheet")
    p_send.add_argument("--creds", default="service_account.json")
    p_send.add_argument(
        "--consent-date", required=True,
        help="Date (YYYY-MM-DD) the client actually signed/returned their written authorization. Required.",
    )
    p_send.set_defaults(func=cmd_send)

```



```
p_consent = sub.add_parser("consent-form", help="Generate the client authorization form to send/sign before any r
p_consent.add_argument("--client-name", required=True)
p_consent.add_argument("--client-email", required=True)
p_consent.add_argument("--target-url", required=True)
p_consent.add_argument("--state", default="FL")
p_consent.set_defaults(func=cmd_consent_form)
```

```
p_overdue = sub.add_parser(
    "check-overdue",
    help="List requests past their deadline and attach registrant/hosting lookup",
)
p_overdue.add_argument("--sheet-id", required=True)
p_overdue.add_argument("--creds", default="service_account.json")
p_overdue.set_defaults(func=cmd_check_overdue)
```

```
p_lookup = sub.add_parser("lookup-owner", help="Look up registrant and hosting info for a URL")
p_lookup.add_argument("--target-url", required=True)
p_lookup.set_defaults(func=cmd_lookup_owner)
```

```
p_states = sub.add_parser("list-states", help="List supported states and their statute data")
p_states.set_defaults(func=cmd_list_states)
```

```
args = parser.parse_args()
args.func(args)
```

```
if __name__ == "__main__":
    main()
```